Bornhuetter-Ferguson Based Actuarial Reserving Tool with Interactive Visualisation

Motivation

Loss reserving stands as one of the most critical functions in actuarial practice, directly impacting an insurer's financial statements, regulatory capital requirements, and strategic decision-making. Actuaries must estimate future claim payments for claims that have been incurred but not yet fully paid, known as Incurred But Not Reported (IBNR) reserves, as well as reserves for reported claims that are still developing.

The complexity of reserving arises from the inherent uncertainty in claim development patterns, the need to project future payments based on incomplete historical data, and the requirement to provide transparent, defensible estimates to stakeholders including regulators, auditors, and senior management.

This project develops a comprehensive reserving tool that implements multiple industry-standard reserving methods, including the Chain Ladder method, the Bornhuetter-Ferguson method, and development factor analysis. By combining rigorous mathematical foundations with an intuitive, interactive dashboard, this tool enables actuaries to perform sophisticated reserve analyses, compare methodologies, and communicate results effectively.

The implementation follows professional software engineering practices, utilising object-oriented design principles to ensure maintainability, extensibility, and reliability. The visual dashboard provides real-time sensitivity analysis, allowing actuaries to explore how reserve estimates respond to changes in development patterns, prior assumptions, and methodological choices.

Theory

This project is based on the fundamental framework of loss triangle analysis, which forms the cornerstone of actuarial reserving practice. We present the mathematical foundations of the principal reserving methods implemented in this tool.

Loss Triangle Structure

Definition (Loss Triangle)

A loss triangle is a triangular array of cumulative or incremental claim amounts, where rows represent accident periods (or origin periods) and columns represent development periods (or lag periods). Let $C_{i,j}$ denote the cumulative claims for accident period i at development period j, where $i, j \in \{0, 1, 2, \ldots, n\}$.
The observed triangle contains values for $i + j \leq n$, representing claims that have been incurred and developed up to the current evaluation date. The lower triangle, where $i + j > n$, contains the unknown future claim amounts that must be estimated.

The fundamental challenge in reserving is to project the unobserved lower triangle based on patterns observed in the upper triangle.

Chain Ladder Method

Theorem (Chain Ladder Development Factors)

The Chain Ladder method assumes that the development pattern is stable across accident periods. For each development period j, the age-to-age development factor is defined as:

$$f_j = \frac{\sum_{i=0}^{n-j-1} C_{i,j+1}}{\sum_{i=0}^{n-j-1} C_{i,j}}, \quad j = 0, 1, \ldots, n-1.$$

The development factors f_j represent the multiplicative factors by which cumulative claims develop from period j to period $j+1$. These factors are computed as weighted averages across all available accident periods, with weights proportional to the cumulative claims at the earlier development period.

Once the development factors are estimated, the projected cumulative claims for the lower triangle are computed recursively:

$$\hat{C}_{i,j} = C_{i,n-i} \prod_{k=n-i}^{j-1} f_k, \quad \text{for } i + j > n.$$

The ultimate claims for accident period i are then:

$$\hat{C}_{i,\infty} = C_{i,n-i} \prod_{k=n-i}^{n-1} f_k.$$

The total reserve is the sum of the incremental claims in the lower triangle:

$$R = \sum_{i=0}^{n} \left(\hat{C}_{i,\infty} - C_{i,n-i} \right).$$

Bornhuetter-Ferguson Method

Theorem (Bornhuetter-Ferguson Reserve)

The Bornhuetter-Ferguson method combines prior expectations of ultimate claims with observed development patterns. For accident period i, let μ_i denote the prior expectation of ultimate claims. The Bornhuetter-Ferguson reserve is:

$$R_i^{BF} = \mu_i \left(1 - \frac{C_{i,n-i}}{\hat{C}_{i,\infty}^{CL}} \right),$$

where $\hat{C}_{i,\infty}^{CL}$ is the Chain Ladder ultimate estimate for accident period i.

The Bornhuetter-Ferguson method provides a compromise between the Chain Ladder method (which relies entirely on observed data) and prior expectations. It is particularly useful when early development periods have limited data or when there are known changes in the business that affect claim emergence patterns.

The method can be interpreted as applying the Chain Ladder development pattern to the unearned portion of the prior ultimate estimate, where the unearned portion is $1 - C_{i,n-i}/\hat{C}_{i,\infty}^{CL}$.

Development Factor Analysis

Definition (Incremental Development Factors)

The incremental development factors measure the proportion of ultimate claims that emerge in each development period. For development period j, the incremental factor is:

$$\lambda_j = \frac{\sum_{i=0}^{n-j}(C_{i,j} - C_{i,j-1})}{\sum_{i=0}^{n-j}\hat{C}_{i,\infty}}, \quad j = 1, 2, \ldots, n,$$

where $C_{i,0}$ represents the initial claims for accident period i.

The incremental factors λ_j sum to unity and provide insight into the speed and pattern of claim emergence. They are essential for understanding the tail development of the portfolio and for validating the reasonableness of reserve estimates.

Statistical Properties and Assumptions

The Chain Ladder method relies on several key assumptions:

1. The development pattern is stable across accident periods.
2. There are no systematic changes in claim handling or reporting practices.
3. The development factors are independent of the accident period.
4. Sufficient historical data exists to estimate reliable development factors.

Violations of these assumptions can lead to biased reserve estimates. The Bornhuetter-Ferguson method provides robustness against such violations by incorporating prior information, but it requires accurate prior estimates of ultimate claims.

Reserve Uncertainty and Validation

While point estimates of reserves are essential, actuaries must also assess the uncertainty in these estimates. Common approaches include:

- Bootstrap methods to generate reserve distributions.
- Mack's model for estimating the standard error of Chain Ladder reserves.
- Sensitivity analysis by varying development factors and prior assumptions.
- Comparison across multiple reserving methods to assess consistency.

Data Requirements

- Historical loss triangle data in cumulative or incremental format (accident period $\times$ development period).
- Prior ultimate claim estimates for Bornhuetter-Ferguson method (optional but recommended).
- Development period labels (typically months, quarters, or years).
- Accident period labels (typically calendar years or policy years).
- The tool includes sample data generators for demonstration purposes, but production use requires actual claims data.

Code

The following code excerpts illustrate production-quality implementations using object-oriented design principles.

Python: Object-Oriented Reserving Engine

```python
import numpy as np
import pandas as pd
from typing import Optional, Tuple, Dict
from abc import ABC, abstractmethod

class LossTriangle:
    """
    Represents a loss triangle with cumulative or incremental claims data.
    """

    def __init__(self, data: np.ndarray, is_cumulative: bool = True):
        """
        Initialise loss triangle.

        Args:
            data: Triangular array of claims data
            is_cumulative: Whether data is cumulative (True) or incremental (False)
        """
        self.data = data
        self.is_cumulative = is_cumulative
        self.n_periods = data.shape[0]

    def to_cumulative(self) -> np.ndarray:
        """Convert incremental triangle to cumulative."""
        if self.is_cumulative:
            return self.data.copy()
        cumulative = np.zeros_like(self.data)
        for i in range(self.n_periods):
            for j in range(self.n_periods - i):
                if j == 0:
                    cumulative[i, j] = self.data[i, j]
                else:
                    cumulative[i, j] = cumulative[i, j-1] + self.data[i, j]
        return cumulative

    def to_incremental(self) -> np.ndarray:
        """Convert cumulative triangle to incremental."""
        if not self.is_cumulative:
            return self.data.copy()
        incremental = np.zeros_like(self.data)
        for i in range(self.n_periods):
            for j in range(self.n_periods - i):
                if j == 0:
                    incremental[i, j] = self.data[i, j]
                else:
                    incremental[i, j] = self.data[i, j] - self.data[i, j-1]
```

```
        return incremental


class ReservingMethod(ABC):
    """Abstract base class for reserving methods."""

    @abstractmethod
    def calculate_reserves(self, triangle: LossTriangle) -> Dict:
        """Calculate reserves using the specific method."""
        pass


class ChainLadderMethod(ReservingMethod):
    """
    Implementation of the Chain Ladder reserving method.
    """

    def __init__(self):
        self.development_factors = None
        self.projected_triangle = None

    def calculate_development_factors(self, triangle: LossTriangle) ->
        np.ndarray:
        """Calculate age-to-age development factors."""
        cumulative = triangle.to_cumulative()
        n = triangle.n_periods
        factors = np.zeros(n - 1)

        for j in range(n - 1):
            numerator = 0.0
            denominator = 0.0
            for i in range(n - j - 1):
                if cumulative[i, j] > 0:
                    numerator += cumulative[i, j + 1]
                    denominator += cumulative[i, j]
            if denominator > 0:
                factors[j] = numerator / denominator
            else:
                factors[j] = 1.0

        return factors

    def project_triangle(self, triangle: LossTriangle) -> np.ndarray:
        """Project the lower triangle using development factors."""
        cumulative = triangle.to_cumulative()
        n = triangle.n_periods
        projected = cumulative.copy()
        factors = self.calculate_development_factors(triangle)

        for i in range(n):
            latest_observed = n - i - 1
            if latest_observed < n - 1:
                current_value = cumulative[i, latest_observed]
                for j in range(latest_observed + 1, n):
                    current_value *= factors[j - 1]
                    projected[i, j] = current_value

        return projected
```

```
104
105     def calculate_reserves(self, triangle: LossTriangle) -> Dict:
106         """Calculate Chain Ladder reserves."""
107         cumulative = triangle.to_cumulative()
108         projected = self.project_triangle(triangle)
109         self.development_factors = self.calculate_development_factors(
                triangle)
110         self.projected_triangle = projected
111
112         n = triangle.n_periods
113         ultimates = projected[:, -1]
114         latest_observed = np.array([cumulative[i, n - i - 1]
115                                     for i in range(n)])
116         reserves = ultimates - latest_observed
117
118         incremental_projected = np.zeros_like(projected)
119         for i in range(n):
120             for j in range(n):
121                 if j == 0:
122                     incremental_projected[i, j] = projected[i, j]
123                 else:
124                     incremental_projected[i, j] = (projected[i, j]
125                                                    - projected[i, j-1])
126
127         return {
128             'ultimates': ultimates,
129             'reserves': reserves,
130             'total_reserve': np.sum(reserves),
131             'development_factors': self.development_factors,
132             'projected_triangle': projected,
133             'incremental_projected': incremental_projected
134         }
135
136
137 class BornhuetterFergusonMethod(ReservingMethod):
138     """
139     Implementation of the Bornhuetter-Ferguson reserving method.
140     """
141
142     def __init__(self, prior_ultimates: np.ndarray):
143         """
144         Initialise with prior ultimate estimates.
145
146         Args:
147             prior_ultimates: Array of prior ultimate estimates for each
                    accident period
148         """
149         self.prior_ultimates = prior_ultimates
150         self.chain_ladder = ChainLadderMethod()
151
152     def calculate_reserves(self, triangle: LossTriangle) -> Dict:
153         """Calculate Bornhuetter-Ferguson reserves."""
154         cl_result = self.chain_ladder.calculate_reserves(triangle)
155         cumulative = triangle.to_cumulative()
156         n = triangle.n_periods
157
158         cl_ultimates = cl_result['ultimates']
159         latest_observed = np.array([cumulative[i, n - i - 1]
```

```
                                    for i in range(n)])

        # Calculate BF reserves
        bf_reserves = np.zeros(n)
        bf_ultimates = np.zeros(n)

        for i in range(n):
            if cl_ultimates[i] > 0:
                development_ratio = latest_observed[i] / cl_ultimates[i
                    ]
                bf_reserves[i] = self.prior_ultimates[i] * (1 -
                    development_ratio)
                bf_ultimates[i] = latest_observed[i] + bf_reserves[i]
            else:
                bf_reserves[i] = self.prior_ultimates[i]
                bf_ultimates[i] = self.prior_ultimates[i]

        return {
            'ultimates': bf_ultimates,
            'reserves': bf_reserves,
            'total_reserve': np.sum(bf_reserves),
            'prior_ultimates': self.prior_ultimates,
            'chain_ladder_ultimates': cl_ultimates,
            'development_ratios': latest_observed / cl_ultimates
        }


class ReservingAnalyser:
    """
    Main class for performing comprehensive reserving analysis.
    """

    def __init__(self, triangle: LossTriangle):
        self.triangle = triangle
        self.results = {}

    def run_chain_ladder(self) -> Dict:
        """Run Chain Ladder analysis."""
        method = ChainLadderMethod()
        self.results['chain_ladder'] = method.calculate_reserves(self.
            triangle)
        return self.results['chain_ladder']

    def run_bornhuetter_ferguson(self, prior_ultimates: np.ndarray) ->
        Dict:
        """Run Bornhuetter-Ferguson analysis."""
        method = BornhuetterFergusonMethod(prior_ultimates)
        self.results['bornhuetter_ferguson'] = method.
            calculate_reserves(self.triangle)
        return self.results['bornhuetter_ferguson']

    def calculate_incremental_factors(self, ultimates: np.ndarray) ->
        np.ndarray:
        """Calculate incremental development factors."""
        incremental = self.triangle.to_incremental()
        n = self.triangle.n_periods
        factors = np.zeros(n)

```

```
        for j in range(n):
            numerator = 0.0
            denominator = 0.0
            for i in range(n - j):
                numerator += incremental[i, j]
                denominator += ultimates[i]
            if denominator > 0:
                factors[j] = numerator / denominator

        return factors
```

This object-oriented design ensures modularity, extensibility, and maintainability. The abstract base class `ReservingMethod` allows for easy addition of new reserving methods, whilst the `ReservingAnalyser` class provides a unified interface for comprehensive analysis.

Python: Streamlit Visual Reserving Dashboard

```
import streamlit as st
import pandas as pd
import numpy as np
import plotly.graph_objects as go
import plotly.express as px
from plotly.subplots import make_subplots
from reserving_engine import (
    LossTriangle, ReservingAnalyser,
    ChainLadderMethod, BornhuetterFergusonMethod
)

st.set_page_config(
    page_title="Actuarial Reserving Tool",
    page_icon="      ",
    layout="wide",
    initial_sidebar_state="expanded"
)

# Custom CSS for professional styling
st.markdown("""
    <style>
    .main-header {
        font-size: 2.5rem;
        font-weight: bold;
        color: #6865F2;
        text-align: center;
        margin-bottom: 1rem;
    }
    .metric-card {
        background-color: #f0f2f6;
        padding: 1rem;
        border-radius: 0.5rem;
        border-left: 4px solid #6865F2;
    }
    </style>
""", unsafe_allow_html=True)

st.markdown('<h1 class="main-header">Actuarial Reserving Analysis
   Dashboard</h1>',
            unsafe_allow_html=True)
```

```
40
41  st.markdown("""
42      This interactive dashboard provides comprehensive loss reserving
           analysis
43      using industry-standard methods including Chain Ladder and
           Bornhuetter-Ferguson.
44      Upload your loss triangle data or use the sample data generator to
           explore
45      the tool's capabilities.
46  """)
47
48  st.markdown("---")
49
50  # Sidebar for inputs
51  with st.sidebar:
52      st.header("       Data Input")
53
54      data_source = st.radio(
55          "Data Source",
56          ["Sample Data Generator", "Upload CSV File"],
57          help="Choose to generate sample data or upload your own"
58      )
59
60      if data_source == "Sample Data Generator":
61          n_periods = st.slider(
62              "Number of Periods",
63              min_value=5,
64              max_value=15,
65              value=10,
66              help="Number of accident and development periods"
67          )
68
69          base_claims = st.number_input(
70              "Base Claims Amount",
71              min_value=1000.0,
72              max_value=100000.0,
73              value=10000.0,
74              step=1000.0
75          )
76
77          volatility = st.slider(
78              "Volatility",
79              min_value=0.1,
80              max_value=0.5,
81              value=0.2,
82              help="Claim development volatility"
83          )
84
85          if st.button("Generate Sample Data"):
86              # Generate sample triangle
87              np.random.seed(42)
88              triangle_data = generate_sample_triangle(n_periods,
                    base_claims, volatility)
89              st.session_state['triangle_data'] = triangle_data
90              st.session_state['n_periods'] = n_periods
91
92      else:
93          uploaded_file = st.file_uploader(
```

```
            "Upload Loss Triangle CSV",
            type=['csv'],
            help="CSV file with loss triangle data"
        )

        if uploaded_file is not None:
            df = pd.read_csv(uploaded_file)
            triangle_data = df.values
            st.session_state['triangle_data'] = triangle_data
            st.session_state['n_periods'] = triangle_data.shape[0]

    st.markdown("---")
    st.header("       Method Selection")

    run_chain_ladder = st.checkbox("Chain Ladder Method", value=True)
    run_bf = st.checkbox("Bornhuetter-Ferguson Method", value=True)

    if run_bf:
        bf_prior_type = st.selectbox(
            "Prior Ultimates Source",
            ["Chain Ladder Ultimates", "Manual Input", "Scaled Chain
               Ladder"],
            help="Source for prior ultimate estimates"
        )

        if bf_prior_type == "Manual Input":
            st.text_input("Enter prior ultimates (comma-separated)")
        elif bf_prior_type == "Scaled Chain Ladder":
            scale_factor = st.slider(
                "Scale Factor",
                min_value=0.5,
                max_value=2.0,
                value=1.0,
                step=0.1
            )

# Main analysis area
if 'triangle_data' in st.session_state:
    triangle_data = st.session_state['triangle_data']
    n_periods = st.session_state['n_periods']

    triangle = LossTriangle(triangle_data, is_cumulative=True)
    analyser = ReservingAnalyser(triangle)

    # Display input triangle
    st.subheader("       Input Loss Triangle")

    triangle_df = pd.DataFrame(
        triangle_data,
        columns=[f"Dev {j}" for j in range(n_periods)],
        index=[f"Acc {i}" for i in range(n_periods)]
    )

    # Create heatmap of input triangle
    fig_triangle = go.Figure(data=go.Heatmap(
        z=triangle_data,
        colorscale='Blues',
        text=triangle_data,
```

```
        texttemplate='%{text:,.0f}',
        textfont={"size": 10},
        showscale=True
    ))

    fig_triangle.update_layout(
        title="Cumulative Loss Triangle",
        xaxis_title="Development Period",
        yaxis_title="Accident Period",
        height=500
    )

    st.plotly_chart(fig_triangle, use_container_width=True)

    # Run analyses
    col1, col2 = st.columns(2)

    with col1:
        if run_chain_ladder:
            st.subheader("       Chain Ladder Results")
            cl_results = analyser.run_chain_ladder()

            st.metric(
                label="Total Reserve",
                value=f"  {cl_results['total_reserve']:,.2f}"
            )

            # Development factors chart
            fig_factors = go.Figure()
            fig_factors.add_trace(go.Bar(
                x=[f"Period {i}" for i in range(len(cl_results['
                    development_factors']))],
                y=cl_results['development_factors'],
                marker_color='#6865F2'
            ))
            fig_factors.update_layout(
                title="Development Factors",
                xaxis_title="Development Period",
                yaxis_title="Age-to-Age Factor",
                height=300
            )
            st.plotly_chart(fig_factors, use_container_width=True)

    with col2:
        if run_bf:
            st.subheader("       Bornhuetter-Ferguson Results")

            # Determine prior ultimates
            if 'bf_prior_type' in locals():
                if bf_prior_type == "Chain Ladder Ultimates":
                    cl_results = analyser.run_chain_ladder()
                    prior_ultimates = cl_results['ultimates']
                elif bf_prior_type == "Scaled Chain Ladder":
                    cl_results = analyser.run_chain_ladder()
                    prior_ultimates = cl_results['ultimates'] *
                        scale_factor
                else:
```

```
                    prior_ultimates = np.ones(n_periods) * base_claims
                        * 2
            else:
                cl_results = analyser.run_chain_ladder()
                prior_ultimates = cl_results['ultimates']

            bf_results = analyser.run_bornhuetter_ferguson(
                prior_ultimates)

            st.metric(
                label="Total Reserve (BF)",
                value=f"  {bf_results['total_reserve']:,.2f}"
            )

    # Comparison visualisation
    if run_chain_ladder and run_bf:
        st.markdown("---")
        st.subheader("       Method Comparison")

        comparison_df = pd.DataFrame({
            'Accident Period': [f"Acc {i}" for i in range(n_periods)],
            'Chain Ladder Reserve': cl_results['reserves'],
            'Bornhuetter-Ferguson Reserve': bf_results['reserves']
        })

        fig_comparison = go.Figure()
        fig_comparison.add_trace(go.Bar(
            name='Chain Ladder',
            x=comparison_df['Accident Period'],
            y=comparison_df['Chain Ladder Reserve'],
            marker_color='#6865F2'
        ))
        fig_comparison.add_trace(go.Bar(
            name='Bornhuetter-Ferguson',
            x=comparison_df['Accident Period'],
            y=comparison_df['Bornhuetter-Ferguson Reserve'],
            marker_color='#5DFFBC'
        ))

        fig_comparison.update_layout(
            title="Reserve Comparison by Accident Period",
            xaxis_title="Accident Period",
            yaxis_title="Reserve Amount",
            barmode='group',
            height=400
        )

        st.plotly_chart(fig_comparison, use_container_width=True)

        # Reserve development over time
        st.subheader("       Reserve Development Pattern")

        incremental = triangle.to_incremental()
        incremental_factors = analyser.calculate_incremental_factors(
            cl_results['ultimates'])

        fig_development = go.Figure()
        fig_development.add_trace(go.Scatter(
```

```
            x=list(range(len(incremental_factors))),
            y=incremental_factors * 100,
            mode='lines+markers',
            name='Incremental Development \%',
            line=dict(color='#6865F2', width=3),
            marker=dict(size=10)
        ))

        fig_development.update_layout(
            title="Incremental Development Factors",
            xaxis_title="Development Period",
            yaxis_title="Percentage of Ultimate",
            height=400
        )

        st.plotly_chart(fig_development, use_container_width=True)

    # Detailed results table
    st.markdown("---")
    st.subheader("       Detailed Results")

    results_df = pd.DataFrame({
        'Accident Period': [f"Acc {i}" for i in range(n_periods)],
        'Latest Observed': [triangle_data[i, n_periods - i - 1]
                            for i in range(n_periods)]
    })

    if run_chain_ladder:
        results_df['CL Ultimate'] = cl_results['ultimates']
        results_df['CL Reserve'] = cl_results['reserves']

    if run_bf:
        results_df['BF Ultimate'] = bf_results['ultimates']
        results_df['BF Reserve'] = bf_results['reserves']

    st.dataframe(results_df.style.format({
        'Latest Observed': '{:,.2f}',
        'CL Ultimate': '{:,.2f}',
        'CL Reserve': '{:,.2f}',
        'BF Ultimate': '{:,.2f}',
        'BF Reserve': '{:,.2f}'
    }), use_container_width=True)

else:
    st.info("       Please configure data input in the sidebar to begin
        analysis.")

def generate_sample_triangle(n_periods: int, base_claims: float,
                             volatility: float) -> np.ndarray:
    """Generate sample loss triangle for demonstration."""
    triangle = np.zeros((n_periods, n_periods))
    development_factors = [1.5, 1.3, 1.2, 1.15, 1.1, 1.08, 1.05, 1.03,
       1.02, 1.01]

    for i in range(n_periods):
        # Initial claims with some randomness
        initial = base_claims * (1 + np.random.normal(0, volatility))
        triangle[i, 0] = max(initial, 0)
```

```
        # Develop claims
        for j in range(1, min(n_periods - i, len(development_factors) +
            1)):
            factor = development_factors[j-1] if j-1 < len(
                development_factors) else 1.01
            factor *= (1 + np.random.normal(0, volatility * 0.3))
            triangle[i, j] = triangle[i, j-1] * factor

    return triangle
```

This dashboard provides a comprehensive, interactive interface for reserving analysis with professional visualisations, method comparison, and detailed reporting capabilities.

Expected Outputs

- Complete reserve estimates using Chain Ladder and Bornhuetter-Ferguson methods.
- Interactive visualisations of loss triangles, development factors, and reserve comparisons.
- Detailed breakdown of reserves by accident period with ultimate claim projections.
- Incremental development factor analysis showing claim emergence patterns.
- Professional dashboard interface suitable for stakeholder presentations and regulatory reporting.
- Exportable results tables and visualisations for documentation and audit purposes.

Interview Questions

1. What are the key assumptions underlying the Chain Ladder method, and how would you validate these assumptions in practice?
2. How does the Bornhuetter-Ferguson method address limitations of the Chain Ladder method, and when would you prefer one method over the other?
3. How would you extend this tool to incorporate uncertainty quantification, such as bootstrap methods or Mack's model for standard errors?
4. What additional reserving methods would you consider implementing, and how would you structure the code to accommodate multiple methods?
5. How would you validate reserve estimates against external benchmarks or industry data?
6. Discuss the challenges of reserving for long-tail lines of business versus short-tail lines, and how the tool could be adapted for each.

Resume Bullets

- Developed an industry-grade actuarial reserving tool implementing Chain Ladder and Bornhuetter-Ferguson methods using object-oriented Python design principles.
- Built an interactive Streamlit dashboard with advanced visualisations for reserve analysis, method comparison, and stakeholder communication.

- Applied actuarial reserving theory to create a production-ready tool capable of handling real-world loss triangle data and generating defensible reserve estimates.
- Designed extensible software architecture enabling easy integration of additional reserving methods and analytical capabilities.